ResearchRAG AI

Final Technical Report

Project: ResearchRAG AI

Theme: Intelligent Academic Research Assistant

This report summarizes the complete project workflow across data preparation, exploratory analysis, supervised modeling, retrieval design, deployment architecture, and real-world impact.

Processed retrieval chunks indexed: 7996

Supervised classification dataset size: 2740

Top categories used for classification: 10

Logistic Regression macro F1: 0.721

MLP Classifier macro F1: 0.769

Prepared for final AI/ML Fellowship submission.

ResearchRAG AI Final Technical Report

ResearchRAG AI

Intelligent Academic Research Assistant

Prepared for the AI/ML Fellowship final submission

1. Executive Summary

ResearchRAG AI is an end-to-end academic assistant built on arXiv metadata. The system combines semantic retrieval, paper classification, and citation-grounded question answering in a Streamlit application. The project addresses a practical research workflow problem: researchers often need to search for relevant papers, understand paper categories, and synthesize evidence-backed answers from a large corpus. Traditional keyword search alone is often insufficient because relevant papers may use different phrasing for related concepts.

The final system integrates four major capabilities:

- semantic search using sentence-transformer embeddings and FAISS
- paper classification from title plus abstract using two local trained models
- citation-grounded question answering with Google Gemini
- fallback mode using local retrieval evidence when live generation is unavailable

The project was deployed as a Streamlit application and packaged with runtime artifacts so it can run without bundling the raw dataset in the deployment repository.

2. Problem Statement

Academic search is often fragmented across multiple tools. A user may need one system for keyword search, another for topic categorization, and yet another for summarization. This leads to workflow friction and weak grounding between search and answer generation. ResearchRAG AI was designed to unify these tasks in one deployable interface with a retrieval-first architecture.

3. Objectives

The final solution was designed to satisfy the following objectives:

- classify papers using title and abstract text
- retrieve semantically similar papers from arXiv content
- answer research questions with evidence from retrieved papers
- provide fallback functionality when the external LLM API is unavailable
- present the workflow through a polished, end-user-facing Streamlit application

4. Dataset And Data Preparation

4.1 Dataset source

- Dataset: arXiv Metadata Snapshot
- Source platform: Kaggle
- URL: https://www.kaggle.com/datasets/Cornell-University/arxiv

4.2 Data snapshot used in the project

Based on the saved EDA and training reports:

- processed records used for retrieval: 5,000
- unique primary categories in processed subset: 122
- supervised dataset size for classification: 2,740
- train split size: 2,192
- test split size: 548
- top categories used for classification: 10

4.3 Exploratory data analysis

EDA showed that the subset is strongly imbalanced. The dominant category is `astro-ph`, followed by physics-heavy classes such as `hep-ph`, `hep-th`, and `quant-ph`. Average title length was 70.43 characters and average abstract length was 802.50 characters, showing that the textual fields were rich enough for both retrieval and classification tasks.

4. Dataset And Data Preparation

The top-10 category setup was selected to reduce sparsity and create a realistic supervised classification problem with sufficient support per class.

4.4 Data pipeline

The project pipeline includes the following stages:

1. load arXiv metadata
2. normalize records and combine title plus abstract for supervised learning
3. generate chunked text for retrieval
4. create sentence embeddings
5. build a FAISS vector index
6. train and evaluate local classification models
7. package runtime artifacts for application deployment

5. System Architecture

ResearchRAG AI uses a retrieval-augmented application design with local ML support:

- Data ingestion and preprocessing from arXiv metadata
- Embedding generation using `sentence-transformers/all-MiniLM-L6-v2`
- FAISS vector index for semantic retrieval
- Local paper classification models saved as `.pkl` pipelines
- Gemini-based answer generation using only retrieved context
- Streamlit frontend for research workflows

At runtime, the application accepts a user query, retrieves relevant text chunks, and then either:

- generates a grounded answer with Gemini, or
- produces a fallback response using retrieved evidence and local classifiers

6. Modeling Approach

6.1 Classification models

Two supervised text classification pipelines were trained:

Logistic Regression
• Features: TF-IDF with unigrams and bigrams
• Max features: 6,000
• Role: strong and interpretable machine learning baseline

MLP Classifier
• Features: TF-IDF with unigrams and bigrams
• Max features: 2,000
• Hidden layer size: 64
• Role: ANN-style comparison model for stronger non-linear classification behavior

6.2 Retrieval model

For semantic search and Q&A context retrieval, the system uses:

• embedding model: `all-MiniLM-L6-v2`
• vector store: FAISS
• persisted index metadata and chunk metadata for efficient deployment

6.3 Generation model

6. Modeling Approach

For research question answering, the system uses Google Gemini with a constrained prompt that instructs the model to:

- use only the provided retrieved context
- avoid invented claims
- cite retrieved sources inline
- state clearly when context is insufficient

7. Evaluation Results

7.1 Classification metrics

The project tracked accuracy, macro precision, macro recall, and macro F1-score.

Logistic Regression
• Accuracy: 0.7938
• Macro Precision: 0.8123
• Macro Recall: 0.6785
• Macro F1: 0.7214

MLP Classifier
• Accuracy: 0.8303
• Macro Precision: 0.8075
• Macro Recall: 0.7480
• Macro F1: 0.7690

7.2 Best model

The MLP classifier achieved the best overall macro F1-score and therefore became the strongest local classifier in the deployed system.

7.3 Interpretation

7. Evaluation Results

The results indicate that:

- both models performed competitively on the top-10-category problem
- the MLP model handled minority-class recall better than the logistic baseline
- macro metrics were important because the dataset remained class-imbalanced

8. Deployment

8.1 Application layer

The system was deployed as a Streamlit application with the following user-facing sections:

- Ask
- Discover
- Classify
- Insights

8.2 Deployment packaging

To make the application deployable without the full raw dataset, the repository includes the runtime artifacts required for inference:

- FAISS index
- embedding metadata
- classification model pipelines
- label encoder
- saved visual exports for EDA

8.3 Secrets and configuration

8. Deployment

Gemini access is controlled through environment variables or Streamlit secrets. When the API key is missing, the system still remains usable through fallback mode.

8.4 Resilience features

The deployed application includes:

- cached loading for retrieval components
- local fallback answer generation
- local classification artifacts
- precomputed retrieval assets

These design choices improve reliability for demos and low-friction cloud deployment.

9. Product Impact

ResearchRAG AI creates value in four practical ways:

9.1 Faster literature discovery

Semantic search helps users find relevant papers even when exact keywords differ from the search query.

9.2 Better research support

Citation-grounded answers reduce the distance between retrieval and synthesis by exposing the evidence used to answer questions.

9.3 Useful classification assistance

Quick category prediction helps users position papers more efficiently, especially when triaging many abstracts.

9.4 Deployment-ready academic assistant

9. Product Impact

The final project is not only a notebook experiment. It is packaged as a usable research tool with a polished interface, saved artifacts, and cloud deployment support.

10. Limitations

The current project still has important constraints:

- retrieval is limited to the processed arXiv subset, not the full corpus
- the supervised classifier focuses on the top 10 categories only
- answer quality depends on the relevance of retrieved chunks
- the fallback response is useful but less fluent and nuanced than live Gemini output
- the deployment repository excludes the raw dataset due to size limits

11. Future Improvements

Potential next steps include:

- expanding category coverage beyond the top-10 setup
- improving retrieval evaluation with ranking-specific metrics
- adding structured citation formatting and source export
- introducing paper comparison views and multi-document summaries
- supporting conversational memory for follow-up research questions
- extending beyond arXiv to additional academic sources

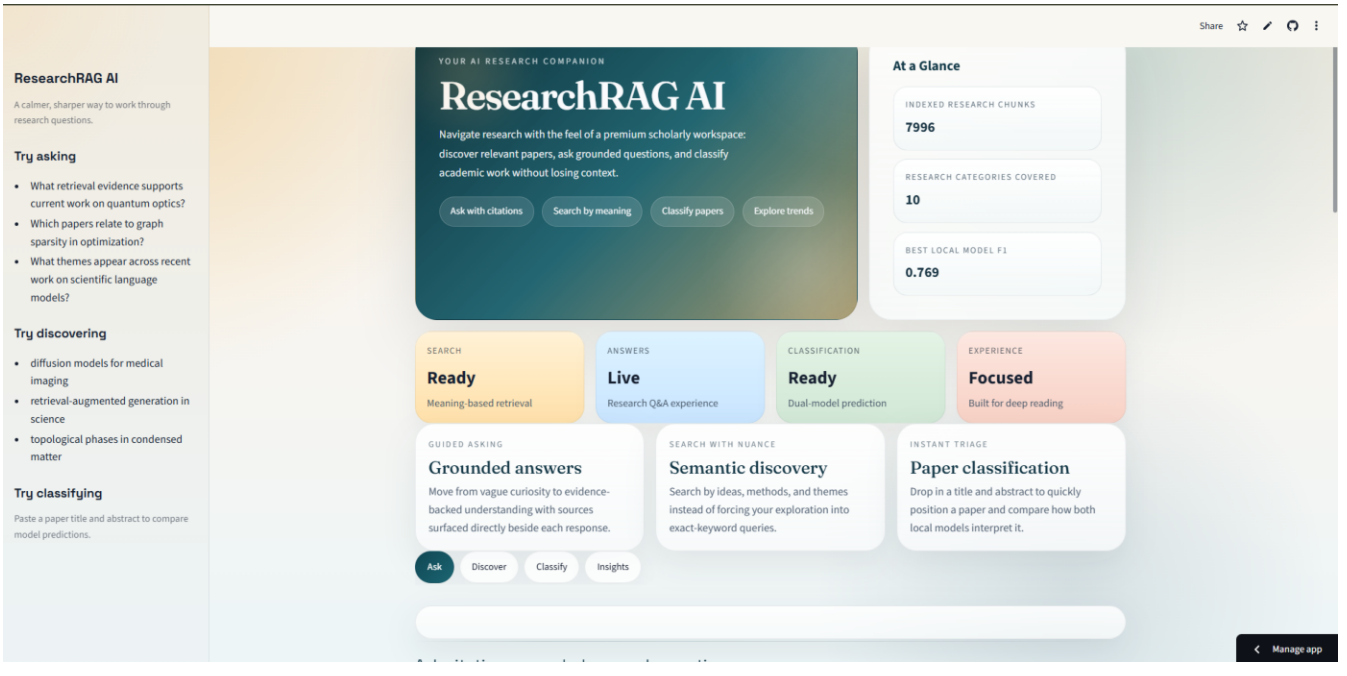
12. Conclusion

ResearchRAG AI successfully combines semantic retrieval, text classification, grounded generation, and deployment engineering into a single academic assistant. The project demonstrates how classical machine learning, vector retrieval, and modern LLMs can be integrated into a practical tool for research exploration. The final system meets the core project objectives and provides a strong foundation for future academic AI applications.

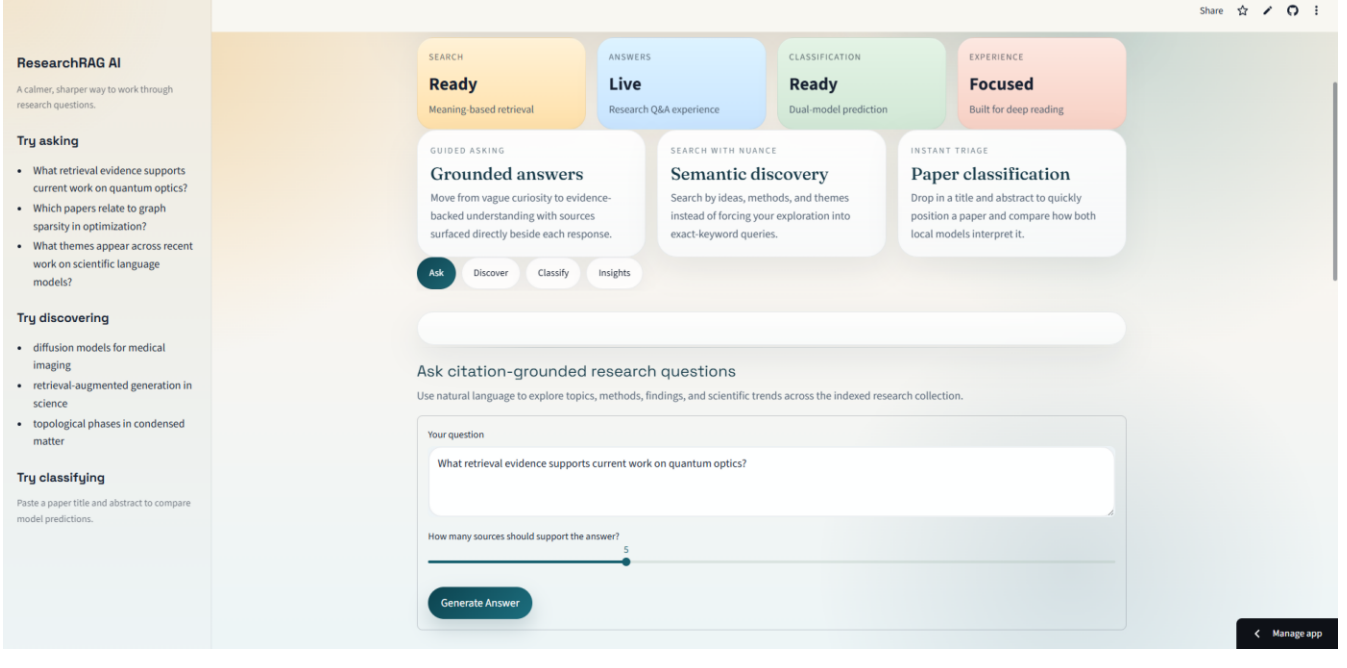
Application Interface

Deployed Streamlit screens showing the premium end-user interface and research Q&A workflow.

home-hero.png



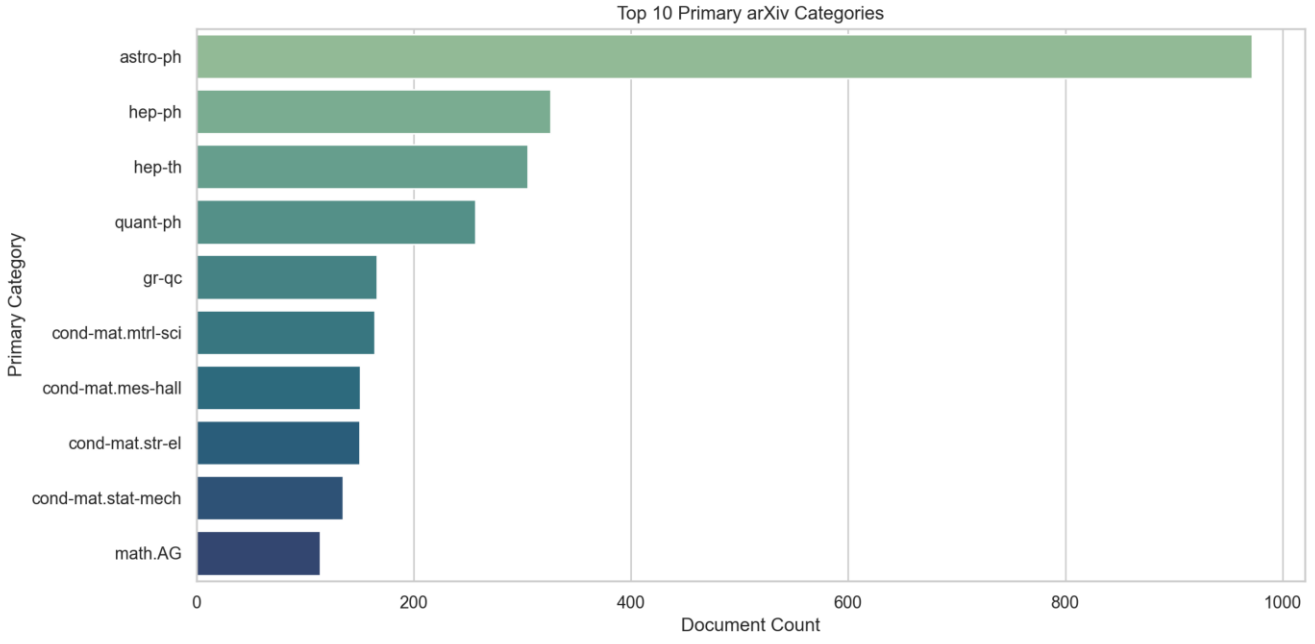
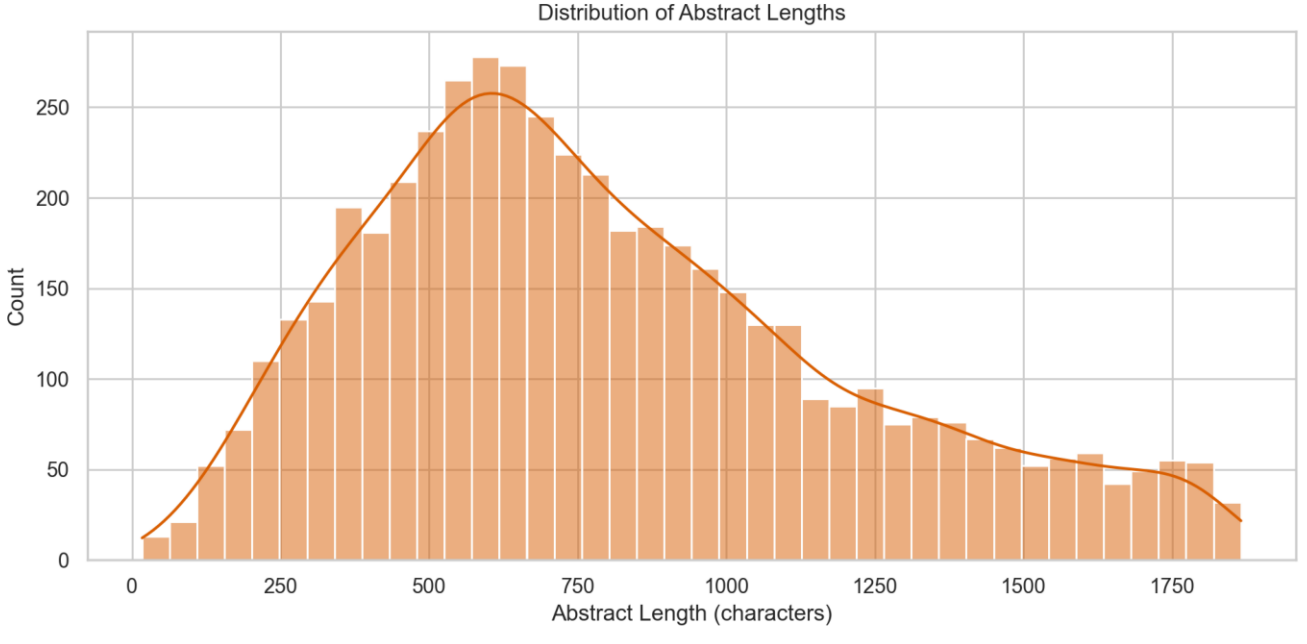
ask-section.png



EDA Visual Evidence

Saved exploratory plots used to understand class distribution and abstract-length behavior in the processed subset.

top_category_distribution.png



abstract_length_distribution.png